

02. Technical Architecture Document

1. System Overview

The MultiScreen platform is a web-based multi-device media display system. The system allows one device to act as the Master Device and multiple smartphones to act as Display Devices. The Master Device creates a session, selects the display layout and media, while connected smartphones display their assigned portion of the selected image or video.

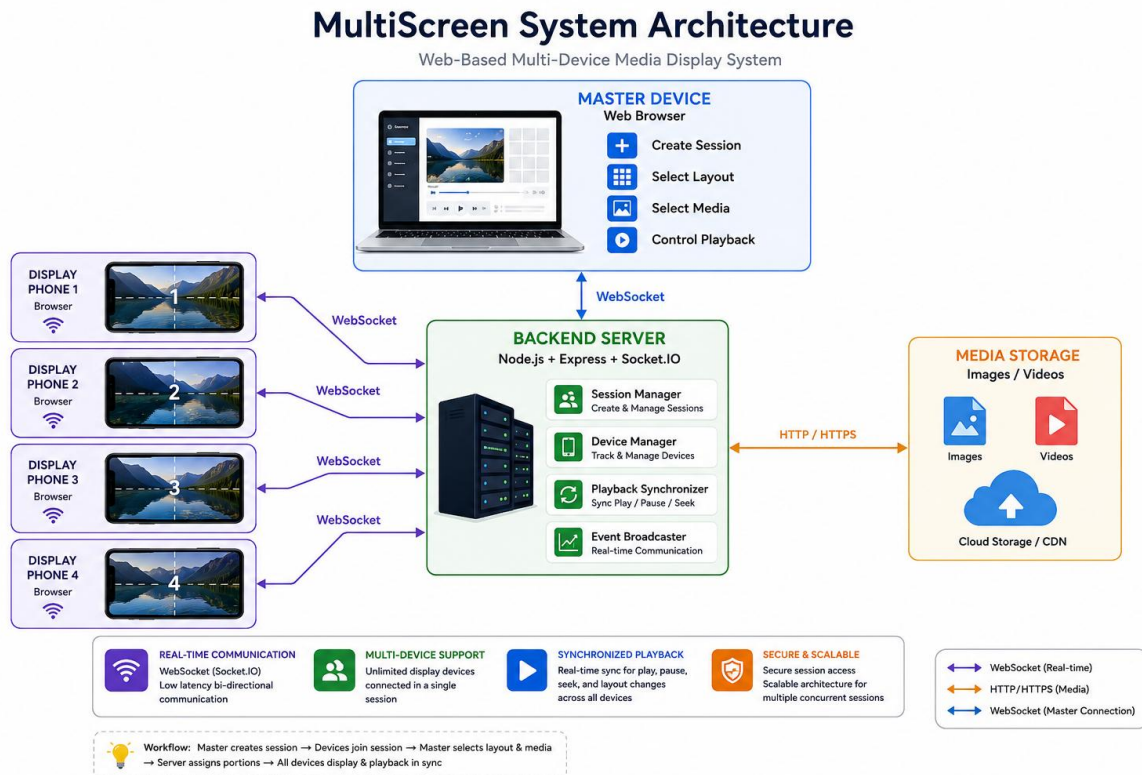
The architecture is designed around three major components:

Frontend Web Application

Backend & Real-Time Session Server

Media Storage

High-Level Architecture



2. Technology Stack

2.1 Frontend Technology Stack

Technology	Purpose	Responsibilities / Benefits
React.js	User Interface & State Management	Master Dashboard, Create/Join Session, Device Management, Layout Selection, Media Library, Upload Interface, Playback Controls, Display Screen, Fullscreen Mode, Connection Status
Vite	Frontend Development & Build Tool	Fast development server, Fast production builds, Simple React configuration, Modern web application support
Tailwind CSS	UI Styling & Responsive Design	Responsive UI, Layouts, Buttons, Cards, Forms, Dashboard Components, Mobile Interface

3. Backend Architecture

3.1 Node.js

Node.js will provide the server-side runtime.

The backend will handle:

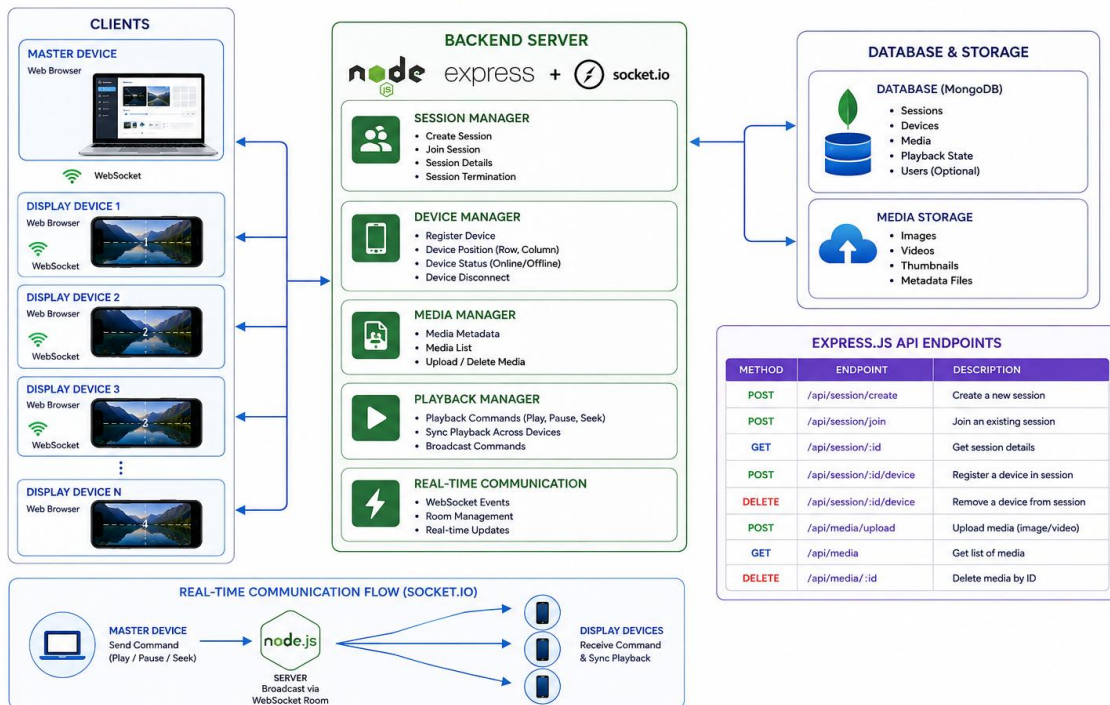
- Session creation
- Session joining
- Device registration
- Device positioning
- Media metadata
- Playback commands
- Real-time communication
- Session termination

3.2 Express.js

- Express.js will provide the HTTP API layer.
- Example API structure:
- POST /api/session/create
- POST /api/session/join
- GET /api/session/:id
- POST /api/session/:id/device
- DELETE /api/session/:id/device
- POST /api/media/upload
- GET /api/media
- DELETE /api/media/:id

Backend Architecture

Node.js + Express.js + Socket.IO Real-time Server



4. Real-Time Communication

Socket.IO

Socket.IO will be used for real-time communication between the Master Device and Display Devices.

This is one of the most important components of the architecture.

Communication Flow

Master Device

|

| PLAY



Socket.IO Server

|

┌───┴───┴───┐



P1 P2 P3 P4

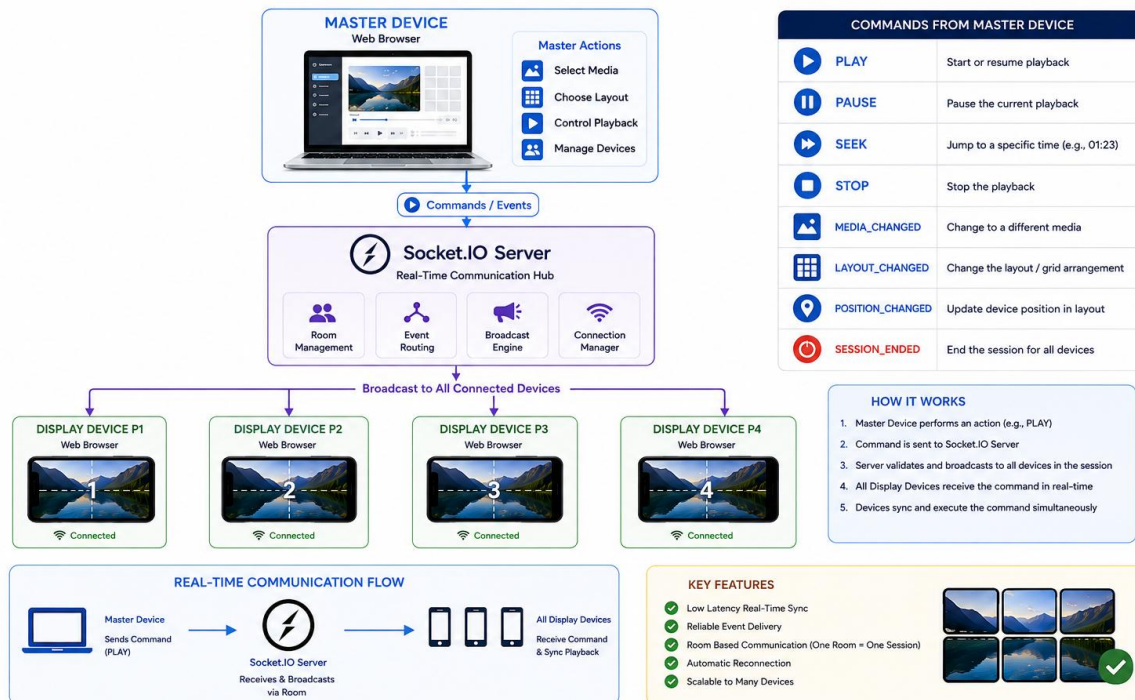
The Master Device can send commands such as:

- PLAY
- PAUSE
- SEEK
- STOP
- MEDIA_CHANGED
- LAYOUT_CHANGED
- POSITION_CHANGED
- SESSION_ENDED

All connected devices receive the appropriate command.

4. Real-Time Communication Architecture

Socket.IO – Master Device ↔ Display Devices



5. Session Architecture

Each viewing session will have a unique Session ID.

Example:

Session ID:

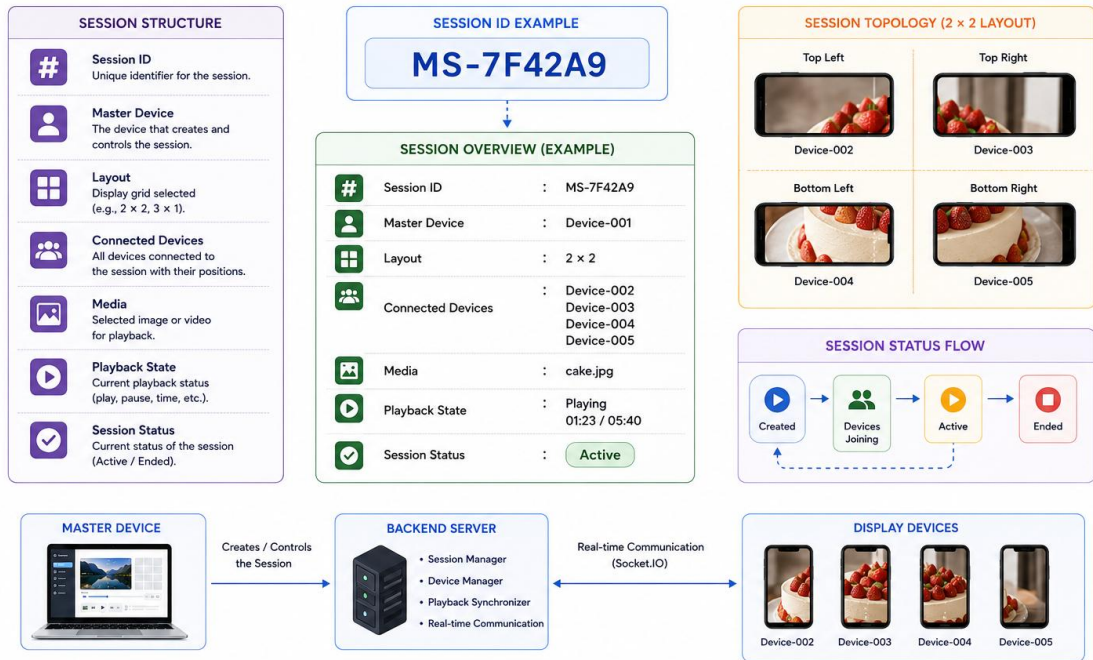
MS-7F42A9

The session contains:

- Session
- Session ID
- Master Device
- Layout
- Connected Devices
- Media
- Playback State
- Session Status

5. Session Architecture

Each viewing session has a unique Session ID and contains all the information required to synchronize multiple devices.



6. QR Code Pairing

The Master Device will generate a QR code containing a session joining URL.

Example: <https://multiscreen.com/join/MS-7F42A9>

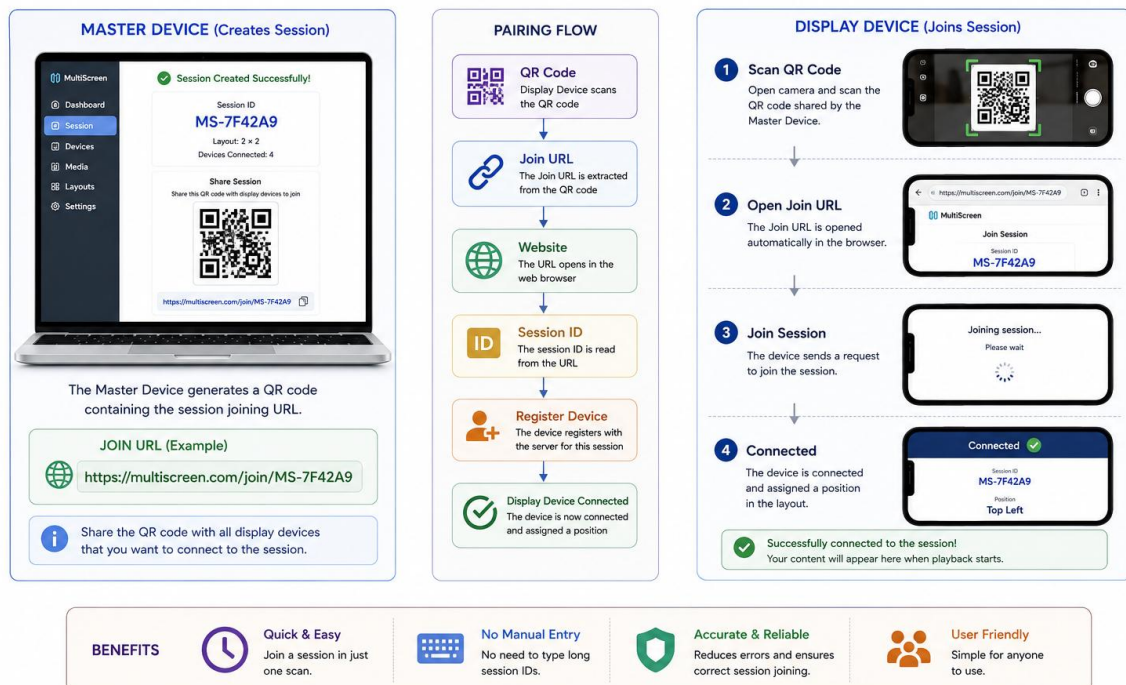
A Display Device scans the QR code.

QR Code -> Join URL -> Website-> Session ID -> Register Device -> Display Device Connected

This eliminates the need for users to manually enter long session IDs.

7. QR Code Pairing

Quick and easy way for display devices to join a session



8. Media Architecture

The system will support two types of media.

8.1 Pre-existing Media

The website will contain a collection of images and videos provided by the platform.

Example: Media Library

Images

└─ Nature

└─ Cakes

└─ Space

└─ Cities

└─ Abstract

Videos

└─ Nature

└─ Animation

└─ Backgrounds

└─ Visual Effects

Users can directly select these files without uploading anything.

8.2 User Uploaded Media

Users can upload their own:

- Images
- Videos

The backend will temporarily store the uploaded media and provide a secure media URL to the connected devices.

Upload Flow

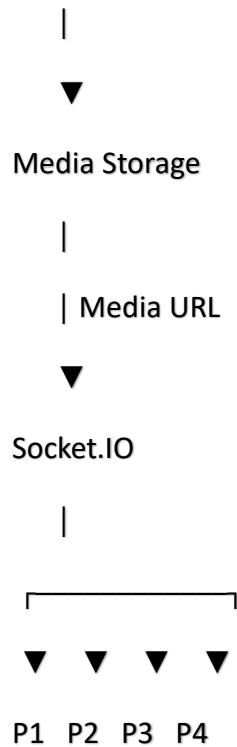
Master Device

|

| Upload

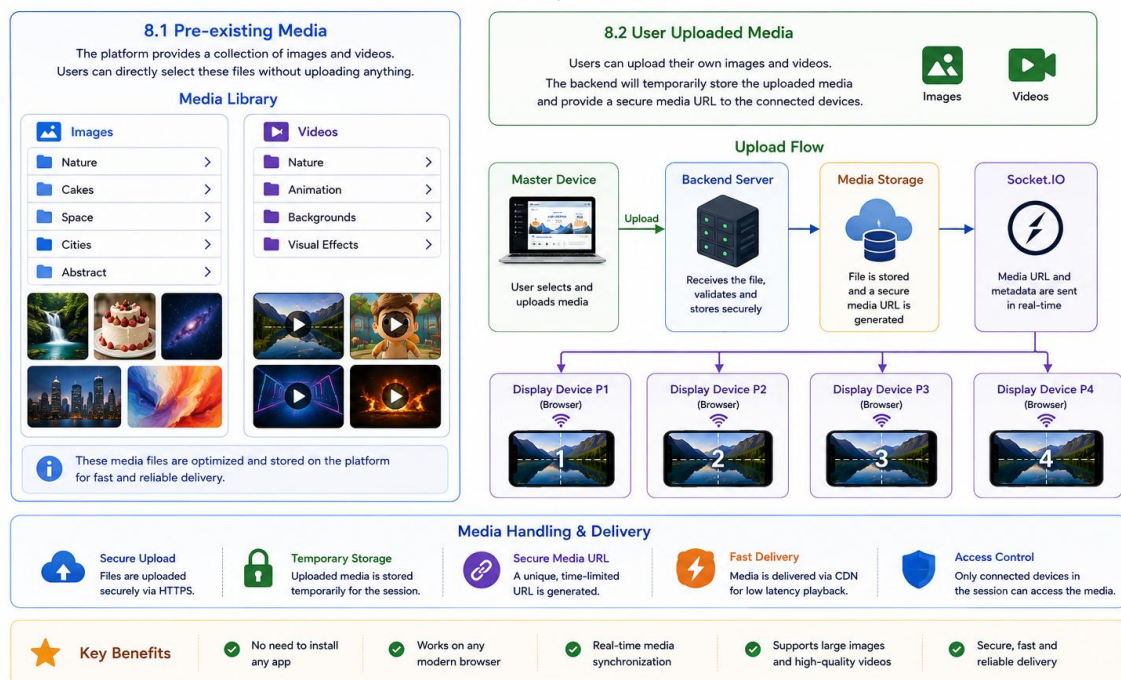


Backend Server



8. Media Architecture

The system supports both pre-existing media from the platform and user uploaded media.



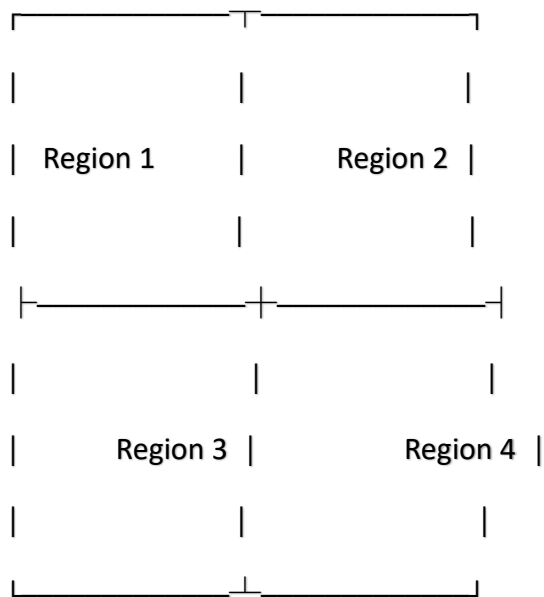
9. Media Rendering Architecture

The system does **not need to physically create four separate video files**.

Instead, each device loads the same media and renders only its assigned region.

For example, with a 2×2 layout:

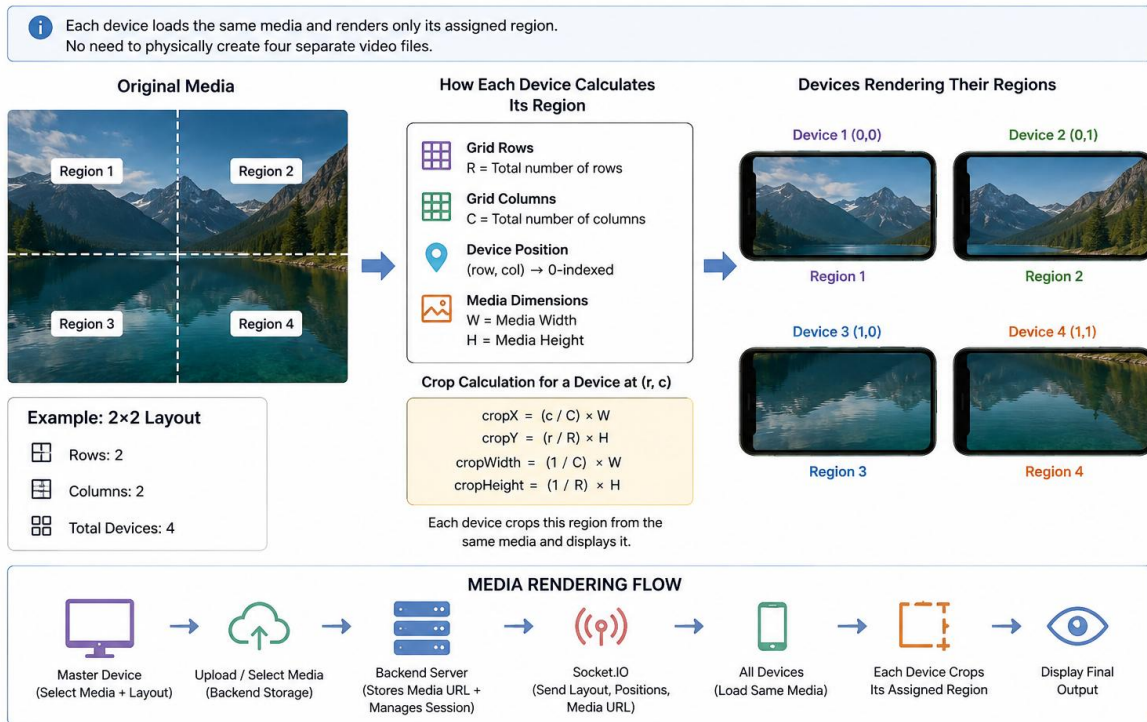
Original Media



The device calculates its crop based on:

- Grid Rows
- Grid Columns
- Device Position
- Media Width
- Media Height

9. MEDIA RENDERING ARCHITECTURE



10. Canvas Rendering

For advanced rendering, the system can use the **HTML5 Canvas API**.

The canvas can render only the required portion of an image or video.

Example:

1. Phone 1 → Render Top-Left region
2. Phone 2 → Render Top-Right region
3. Phone 3 → Render Bottom-Left region
4. Phone 4 → Render Bottom-Right region

This makes the system flexible for different layouts.

10. CANVAS RENDERING

Render only the required portion of the image/video on each device

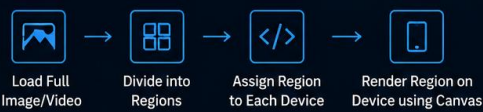
ORIGINAL MEDIA



CANVAS RENDERING

Each device renders only its assigned region using HTML5 Canvas API

HOW IT WORKS



PHONE 1

Render Top-Left region



PHONE 2

Render Top-Right region



PHONE 3

Render Bottom-Left region



PHONE 4

Render Bottom-Right region



BENEFITS

- ✓ No need to create multiple files
- ✓ Real-time flexibility for any layout
- ✓ Optimized performance for all devices
- ✓ Works for images and videos

13. Database Architecture

A database can be used for persistent application data.

Recommended database:

PostgreSQL

The database may contain:

Sessions

sessions

- id
- session_code
- master_device_id
- layout
- status
- created_at
- expires_at

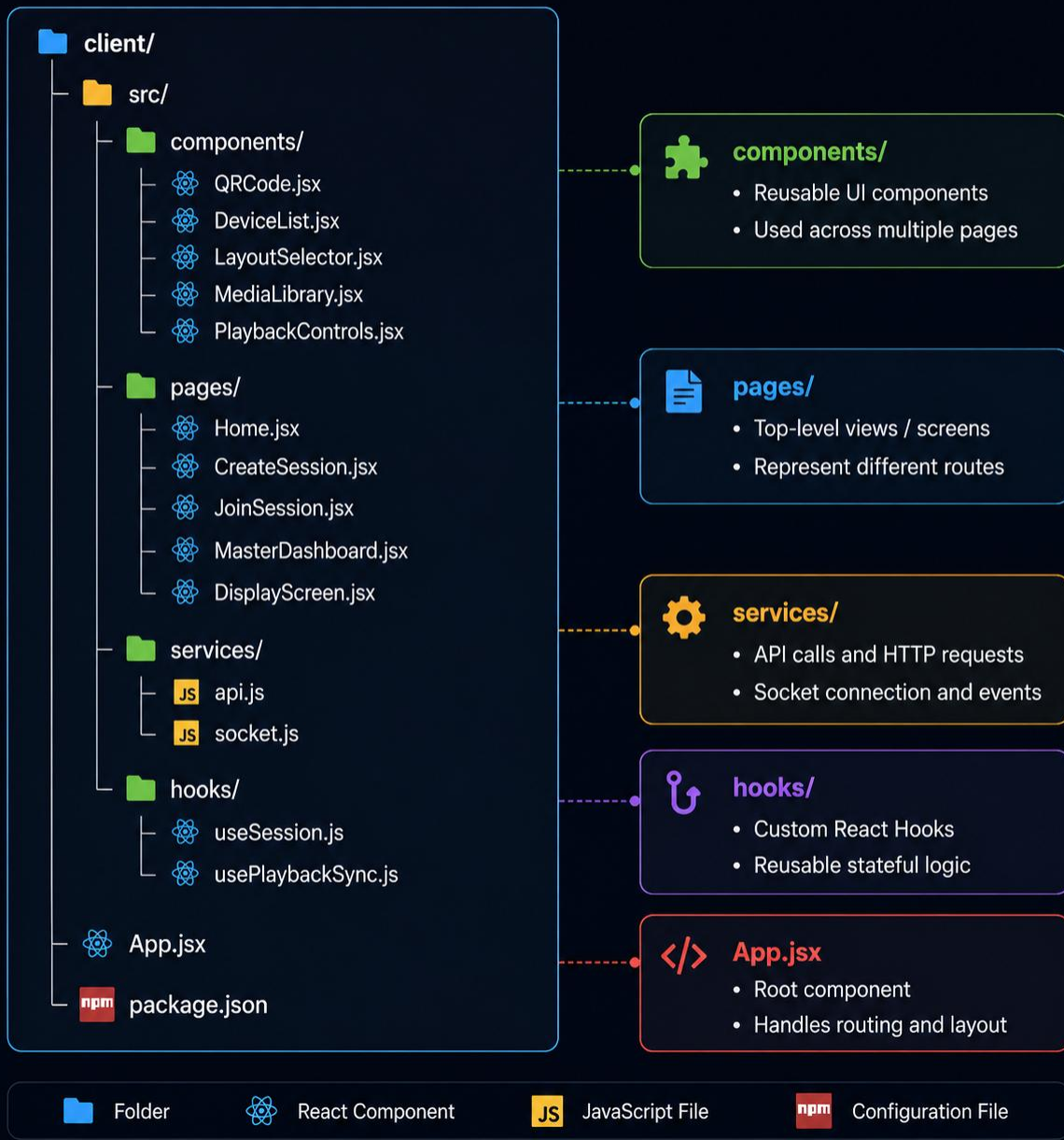
Devices

- id
- session_id
- device_code
- role
- position
- status
- connected_at

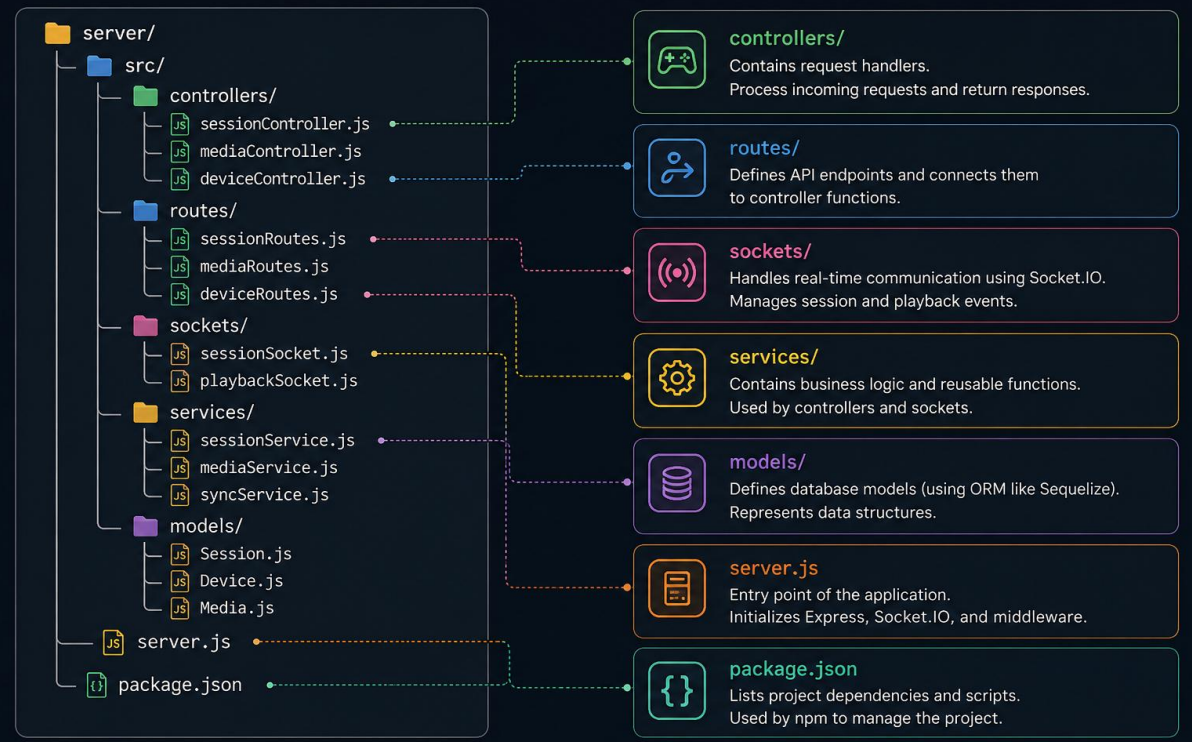
Media

- id
- name
- type
- url
- thumbnail_url
- size
- duration
- created_at

16. Recommended Frontend Structure



15. Recommended Backend Structure



19. PRODUCTION ARCHITECTURE

